European Commission

DPP Registry Semantic Data Guide

DG GROW G2/B3

Last published 2026-10-30 v1.0.



Table of Contents

Chapter 1. Introduction 3
1.1 About this guide 3
Target Audience 3
Scope and Coverage 3

Chapter 2. Solution Overview 4
2.1 Semantic architecture 4
2.2 Semantic artifact inventory 5
2.3 Versioning 6
Semantic pointer 7
Releases representation 8

Chapter 3. DPP Taxonomies 10
3.1 Taxonomy hierarchy 11
3.2 Core vocabulary 11
3.3 Product Group vocabulary 12

Chapter 4. SHACL Shapes 14
4.1 SHACL shapes structure 14
4.2 Constraint types 15

Chapter 5. Data Models 18
5.1 XSD 18
5.2 JSON schema 19
5.3 JSON-LD context 21

Chapter 6. JTC 24 Alignment 23

Chapter 7. Battery DPP Examples 24

Chapter 1. Introduction

1.1 About this guide

This guide documents the semantic artifact inventory underpinning the Digital Product Passport (DPP) and product data ecosystem. The aim is to give a single reference for how meaning, structure, and format fit together across the entire DPP and battery data model.

Target Audience

This guide is intended for EO developers, system integrators, DevOps engineers and security teams responsible for automated DPP Registry integrations.

Scope and Coverage

This document provides relevant information and instructions regarding:

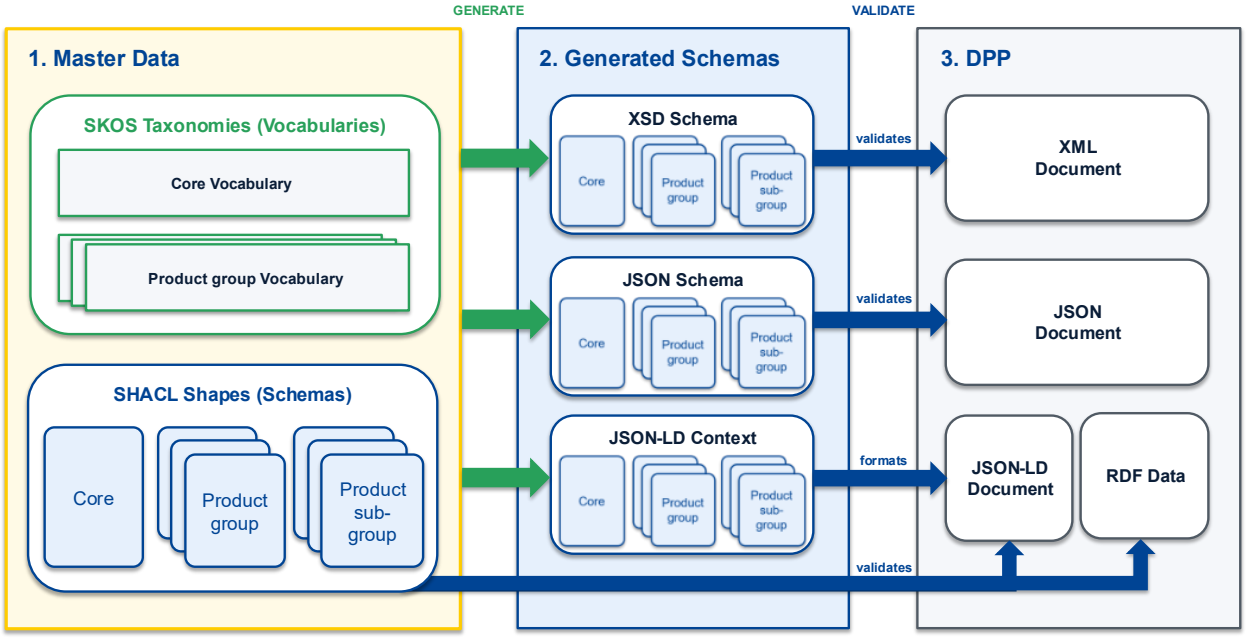
- Each layer of the framework in turn: the SKOS taxonomies that define shared vocabulary and meaning, the SHACL shapes that validate RDF data against that vocabulary, and the format-level artifacts - XSD, JSON Schema, and JSON-LD context - that carry the same structure and semantics into XML, JSON and JSON-LD exchange.
- How its artifacts are structured, how they compose into a layered hierarchy (core, product group, sub-group), and how they relate to one another - so that a single DPP can be represented consistently as RDF or JSON-LD, XML, or JSON, always tracing back to the same underlying model.
- The framework's alignment with the CEN-CENELEC JTC 24 standard (EN 18223:2026), including the one point where this implementation departs from the standard's normative examples.

Chapter 2. Solution Overview

This section outlines the architecture of the Digital Product Passport (DPP) definition and validation framework, designed in compliance with the CEN/CLC/JTC 24[1] standard to guarantee data integrity, interoperability, and semantic consistency across diverse supply chain ecosystems.

2.1 Semantic architecture

The primary objective of the framework is to govern how passport documents are serialized and validated, ensuring that every DPP data point matches rigorous Ecodesign for Sustainable Products Regulation (ESPR)[2] and the correspondent product Regulation (e.g. Batteries and Waste Batteries Regulation[3]) expectations regardless of its final data format.



The Master Data

The entire validation framework is built on a single source of truth model designed to enable technical and semantic interoperability with all Digital Product Passport (DPP)s. Under this architecture, the SKOS Taxonomies and SHACL Shapes act as the master data, governing all downstream schemas and rules:

- SKOS Taxonomies: Semantic classifications establishing the standardized vocabularies, reference terms, and hierarchical relationships required for product categorization.
- SHACL Shapes: Data structure, cardinality, and validation constraints. These are partitioned into product groups (e.g. Battery, Textile, etc.) and, if needed, product sub-groups (e.g. in Batteries: Electric Vehicle Battery – EV, Industrial Battery – IND and Light Means of Transport Battery – LMT). Within each product group and sub-group, the shapes could be further split into views.

Generated Schemas

[1] CEN/CLC/JTC 24 - Digital Product Passport
[2] Regulation (EU) 2024/1781 of the European Parliament and of the Council stablishing a framework for the setting of ecodesign requirements for sustainable products.
[3] Regulation (EU) 2023/143 of the European Parliament and of the Council concerning batteries and waste batteries.

To prevent manual design drift and eliminate configuration errors, all downstream schema files are compiled from the master SHACL shape files. As the master shapes are split into views, JSON schema and XSD artefact could be likewise split into views.

The schemas generated are:

- XML Schema Definitions (XSD) to enforce structural rules on XML serializations.
- JSON Schemas to validate key-value structures in JSON documents.
- JSON-LD Contexts to map standard JSON payloads directly to their semantic triples.

DPP

When a target DPP document is submitted under the CEN/CLC/JTC 24 standard, it is processed through format-specific validation engines to certify compliance before ingestion:

- XML Document: Validated structurally using the compiled XSD schema.
- JSON Document: Validated structurally using the compiled JSON Schema.
- JSON-LD and RDF Documents: Validated semantically. Because these formats express graph-based data, they bypass flat schemas and are validated directly against the master SHACL Shapes (utilizing the corresponding product and product categories rulesets) to verify rich relationship properties and data types.

By decoupling serialization formats from the underlying master rules, the JTC 24 validation framework allows developers to use the data format that best suits their application context while guaranteeing identical validation compliance across the board. The registration process will be requesting JSON data format, as it is the only mandatory data format in accordance with EN 18216:2026.

2.2 Semantic artifact inventory

This inventory catalogues the semantic and technical artifacts that make the interoperability within the Digital Product Passport (DPP) ecosystem possible.

The Master Data

The SKOS taxonomies are built using the Simple Knowledge Organization System (SKOS)[4], these taxonomies organize concepts into structured hierarchies of terms, complete with multilingual labels and defined relationships (broader, narrower, related), so that all systems producing or consuming DPP data draw on a common, unambiguous vocabulary.

Name	Type	Description
Core vocabulary	SKOS vocabulary	It defines core concepts and categories used across Digital Product Passport.
Common vocabulary	SKOS vocabulary	It defines shared concepts and categories that are used identically across product group and sub-groups.
Product group vocabulary	SKOS vocabulary	A domain-specific SKOS vocabulary scoped to product-related concept under the correspondent Regulation.
Product sub-group vocabulary	SKOS vocabulary	A domain-specific SKOS vocabulary scoped to product sub-group related concept under the correspondent Regulation.

Within these taxonomies, different vocabularies illustrate how the SKOS taxonomies operationalize semantic consistency. They are introduced here in detail:

[4] SKOS Simple Knowledge Organization System Reference

Name	Type	Description
Auxiliar core vocabularies	SKOS vocabulary	A set of controlled vocabularies referenced from the Core Vocabulary and used through all products groups and sub-groups. E.g.: Granularity level, Measurement unit, Critical raw material, etc.
Product sub-group vocabulary	SKOS vocabulary	The product sub-group to which the product groups belongs to. It is referenced in the Product group vocabulary. Examples in Battery product group: EV (Electric Vehicle), IND (Industrial), or LMT (Light Means of Transport).

Shapes Constraint Language (SHACL)[5] files that define validation rules for RDF data - specifying required properties, cardinalities, datatypes, and value constraints that a DPP or product dataset must satisfy to be considered conformant. SHACL shapes act as the "rulebook" checking that actual data graphs correctly and completely implement the underlying data model.

Name	Type	Description
Core shapes	SHACL shapes	Defines the baseline validation rules applicable to any Digital Product Passport, regardless of product type. This includes generic, and cross-cutting requirements.
Product group shapes	SHACL shapes	Defines the validation rules common to a specific product (e.g. batteries), regardless of their product sub-group. This includes requirements shared across every product group. It extends the generic DPP shape with product-specific constraints, and acts as the common parent that the product sub-group shapes. Product group shapes could be split into views.
Product sub-group shapes	SHACL shapes	Defines the validation rules specific to a product sub-group. Product sub-group shapes could be split into views.

Generated Schemas

The semantic artifacts include format-level artifacts that govern how DPP and product data is structured and exchanged in practice: XSD, JSON Schema, and JSON-LD context files.

Name	Type	Description
Core, Product group and Product sub-group XSDs	XSD	Defines the structure, element hierarchy and datatypes specific to product group or sub-group. Product group and product sub-group XSDs could be split into views.
Core, Product group and Product sub-group JSON Schema	JSON schema	Defines required fields, data types and value constraints specific to product group or sub-group. Product group and product sub-group JSON schema could be split into views.
Core, Product group and Product sub-group JSON-LD context	JSON-LD context	It maps the plain fields used in JSON data to the concepts and terms defined in the SKOS taxonomies.

2.3 Versioning

The semantic artefacts uses two distinct, complementary versioning mechanisms: artifact versioning and release versioning. Both follow Semantic Versioning (SemVer[6]), using the standard X.Y.Z (major.minor.patch) format:

- **X - Major**: a breaking change (e.g. removing a mandatory field, changing a datatype, or altering a constraint in a way that invalidates previously conformant data).
- **Y - Minor**: a backward-compatible addition (e.g. a new optional field, a new concept added to a

[5] Shapes Constraint Language (SHACL)
[6] http://semver.org/

taxonomy).

- **Z - Patch**: a backward-compatible fix (e.g. a corrected definition, label, or source reference, with no structural or semantic impact).

Artifact versioning

Every individual artifact in the inventory - a SKOS taxonomy, a SHACL shape, an XSD, a JSON Schema, or a JSON-LD context - carries its own version. This tracks changes at the level of a single file: when a specific vocabulary or schema is updated, its version is incremented, allowing precise tracking of what changed, when, and at which layer (core, product group, or product sub-group).

Semantic pointer

Versioning is not just recorded as metadata inside each artifact - it is embedded directly into the URI structure and namespaces used to publish and reference artifacts and releases in the repository. This means a version is never ambiguous or separately tracked from the resource itself: the version is part of the identifier.

Artifact-level URIs

Individual artifacts are addressed with a more granular pattern, identifying the product group, the artifact type, the specific artifact, and its own independent version:

- **Product version**:

https://repository.product-passport.ec.europa.eu/product/{product-name}/{type}/{artefact-name}/{SemVer}

- **Sub-group version**:

https://repository.product-passport.ec.europa.eu
/product/{product-name}/subgroup/{subgroup-name}/{type}/{artefact-name}/{SemVer}

For example, SHACL Shapes of Electric Vehicle Battery's subgroup would have this URI:

https://repository.product-passport.ec.europa.eu/product/batteries/subgroup/evb/shapes/evb-shapes/1.0.0

Release-level URIs

Each layer of the hierarchy has its own release URI pattern, with the release's SemVer version as the final path segment:

- **Core release**: https://repository.product-passport.ec.europa.eu/core/releases/{SemVer}
- **Product release**: https://repository.product-passport.ec.europa.eu/product/{product-name}/releases/{SemVer}
- **Sub-group release**: https://repository.product-passport.ec.europa.eu/product/{product-name}/subgroup/{subgroup-name}/releases/{SemVer}

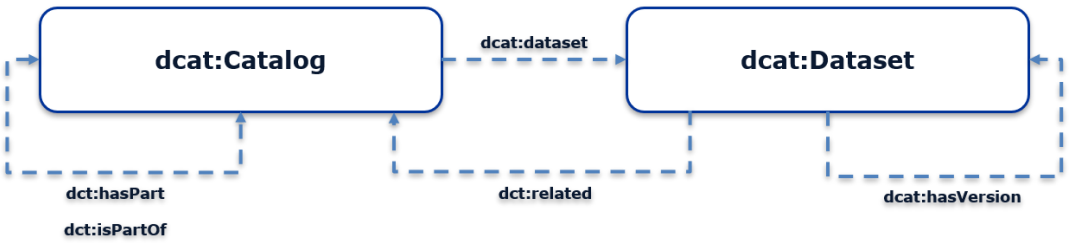
Releases representation

The repository will publish its content as a structured, machine-readable catalogue based on DCAT-AP[7] (Data Catalogue Vocabulary Application Profile for sharing information about Catalogues containing Datasets and Data Services descriptions in Europe). This approach lets users, tools and automated systems discover what each release contains, how its parts relate to one another, and which versions are available, without needing to browse the repository manually.

DCAT-AP will be used to represent the following types of content:

- **Core releases**: the foundational resources shared across the repository.
- **Product releases**: collections of resources that apply to a specific group of products.
- **Product subgroup releases**: more specific subdivisions within a product group.
- **Artefacts**: the individual files and resources contained in any of the above.

The DCAT-AP model is organised as a hierarchy. A single catalogue acts as the entry point, and datasets nested beneath it represent the different levels of content:



The minimum DCAT-AP classes used are the Catalog and Dataset:

DCAT-AP class	Description
dcat:Catalog	It describes the repository as a whole and gives access to everything published within it. In this case, the catalogue is available at: https://repository.product-passport.ec.europa.eu/
dcat:Dataset	It is used to represent each of the following: • Core artefacts • Product artefacts • Product subgroups artefacts Using the same class at every level keeps the model simple and consistent, while the relationships described below make the hierarchy explicit.

Additionally, the relationship between an artefact and the product group or category it belongs to is expressed using two complementary properties from Dublin Core Terms[8]:

DCAT-AP properties	Description
dct:hasPart	It is used on the parent (for example, a product group) to list the artefacts or subgroups it contains.
dct:isPartOf	It is used on the child (for example, an artefact) to point back to the product group or subgroup it belongs to.

[7] DCAT-AP 3.0.1
[8] DCMI: DCMI Metadata Terms

Providing both directions means that users can navigate the hierarchy from either end, whether they start from a product group and want to find its artefacts, or start from an artefact and want to find its context.

Relationships between a product group version and a product subgroup version (or the reverse) are expressed using this property:

DCAT-AP properties	Description
dct:related	It indicates a general association between two resources, where neither strictly depends on the other.

Every release is versioned, and the catalogue captures this information explicitly so that users can identify and retrieve the right version.

DCAT-AP properties	Description
dcat:hasVersion	It lists all the release versions that are available for a resource. It contains one URI per version.

Chapter 3. DPP Taxonomies

Every taxonomy in DPP follows the same modelling pattern, built on the SKOS (Simple Knowledge Organization System) standard. Understanding this shared pattern once makes every taxonomy in DPP readable, regardless of its specific domain.

Element	Description
Concept Scheme	Each taxonomy is published as a single skos:ConceptScheme - the container that groups all of its concepts together and carries scheme-level metadata: a title, a definition, its regulatory source, its creation date, and a lifecycle status. This is the "cover page" of the vocabulary - it describes what the whole taxonomy is for.
Concept	Each individual term in a taxonomy is a skos:Concept. A concept represents one well-defined idea - a data field, a category, a status value - never a raw label or a code by itself. Concepts are what other artifacts (SHACL shapes, JSON Schema, JSON-LD contexts) point back to when they need to reference a standardized meaning.
Notation	Every concept carries a skos:notation - a short, stable, machine-readable code used as the actual field name or value in schemas and data instances. This is the identifier developers and systems actually use, distinct from the human-readable label.
Preferred Label	The skos:prefLabel gives the concept's human-readable name, in a specific language (e.g. @en). This is what a person reading documentation or a user interface sees, as opposed to the notation a machine sees.
Definition	The skos:definition gives a precise, unambiguous explanation of what the concept means. This is the anchor that prevents different systems from silently interpreting the same field differently.
Scheme membership	skos:inScheme (and, for root-level terms, skos:topConceptOf) ties each concept back to its parent Concept Scheme, making explicit which taxonomy a term belongs to - important once multiple taxonomies start referencing each other.
Source	dct:source records the specific regulatory or standards provision that mandates or defines the concept - a regulation article, an annex, or a standard's table number. This is what makes each term legally and technically traceable, not just descriptive.
Status	euvoc:status records the concept's lifecycle state (e.g., CURRENT), following the EU Publications Office's vocabulary status authority table. This lets the taxonomy evolve over time - adding, deprecating, or superseding terms - without breaking systems that reference stable identifiers.
Version info	owl:versionInfo is a simple annotation property carrying a human-readable version label as a literal - typically the SemVer string itself (e.g. "2.1.0").
Version IRI	owl:versionIRI is a stronger, structural annotation that assigns a distinct, dereferenceable IRI to a specific version of an ontology or vocabulary. It the IRI described in the versioning section.

Together, all these elements form a consistent scheme that every taxonomy in DPP taxonomies shares.

Additionally, all taxonomies declare the namespaces following the URI structure pattern:

- Core URI:

```
https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary#
```

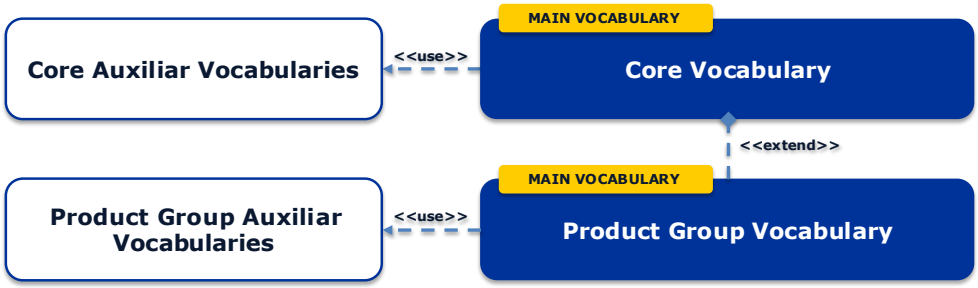
- Product and subgroup URI:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/vocabs/{name}-vocabulary#

https://repository.product-passport.ec.europa.eu/product/{product-name}/subgroup/{subgroup-name}/vocabs/{subgroup}-vocabulary#
```

3.1 Taxonomy hierarchy

This figure presents the vocabulary layer of the DPP semantic framework, showing how the SKOS taxonomies are formally structured and related to one another as vocabulary classes.



At the top, the **Core Vocabulary** acts as the root vocabulary, defining core cross-cutting concepts and using, if needed, the **Auxiliar Vocabularies** (e.g. Granularity-level vocabulary). The **Product Group Vocabulary** extends this root vocabulary, adding product concepts, using Auxiliar Vocabularies.

The diagram uses standard UML notation to make these relationships explicit: solid arrows with hollow heads represent **generalization** ("extends"), showing the inheritance hierarchy from the general DPP vocabulary down to product sub-group-specific ones, while dashed arrows represent **dependency** (“use”), showing where one vocabulary references terms defined in another.

3.2 Core vocabulary

The **Core Vocabulary** is the core taxonomy, published as the concept scheme *core-vocabulary*. It defines the baseline set of attributes common to *any* Digital Product Passport, independent of product type fields.

The scheme’s own metadata already illustrates the shared pattern above: it carries a title and definition, a source pointing to the ESPR regulation itself, a creation date, and a CURRENT status - establishing it as the actively maintained foundation of the whole taxonomy hierarchy.

It defines top-level concepts, covering these broad areas:

- **Passport identity and versioning**: *digitalProductPassportId*, *uniqueProductIdentifier*, *dppStatus*, *lastUpdate*.
- **Scope**: granularity, defining at what level (item, batch, model) the passport applies. The concept granularity are defined in a separate Granularity-Level Vocabulary (*granularity-level*).
- **Compliance referencing**: *contentSpecificationIds*, linking the passport to the specifications it must satisfy.

The *contentSpecificationIds* property carries a set of restrictions enforcing that a DPP instance references the complete chain of releases it was built against, from the core layer up through its product layer and, where applicable, its sub-group layer.

At least two values, referencing a core release and a product release.

- **Core release reference**. At least one of the values must match core release pattern:

```
https://repository.product-passport.ec.europa.eu/core releases/{SemVer}
```

- **Product release reference**. At least one of the values must match product release pattern:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/releases/{SemVer}
```

- **Subgroup release reference** (optional). Optionally, if the DPP belongs to a product subgroup, this

value is expected.

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/subgroup/{subgroup-name}/releases/{release-number}
```

Every one of these concepts is sourced either from **EN 18223:2026** (the JTC 24 technical standard) or directly from **Regulation (EU) 2024/1781** (the ESPR), which is what gives the vocabulary its dual legal-and-technical grounding – a point developed further below.

The Core vocabulary has exactly one *skos:ConceptScheme* (*core-vocabulary*) declared once at the top of the file. Every other entry in the file is a *skos:Concept*, and each of them points back to that single scheme via *skos:inScheme*.

```
@prefix dpp: <https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary#> .

<https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary> a
    owl:Ontology, skos:ConceptScheme;
  owl:versionIRI <https://repository.product-passport.ec.europa.eu/core/vocabs/core-
vocabulary/1.0.0>;
  skos:notation "core-vocabulary";
  skos:prefLabel "Digital Product Passport Core Vocabulary"@en;
  dct:title "Digital Product Passport Core Vocabulary"@en;
  skos:definition "Vocabulary of core concepts used by digital product passport data model."@en;
  dct:source <http://data.europa.eu/eli/reg_impl/2026/1778/oj>,
<http://data.europa.eu/eli/reg/2024/1781/2024-06-28>;
  dct:references
<https://standards.cencenelec.eu/ords/f%3Fp%3DCEN:110:::::FSP_PROJECT%2CFSP_ORG_ID:80714%2C3342699%
26cs%3D12990D18E0F87D7768D50B938A2F4A4D5>;
  dct:created "2026-10-30"^^xsd:date;
  owl:versionInfo "1.0.0";
  skos:hasTopConcept dpp:digitalProductPassportId, dpp:uniqueProductIdentifier, dpp:granularity,
dpp:dppStatus, dpp:lastUpdate, dpp:facilityId,
    dpp:contentSpecificationIds, dpp:resourceTitle, dpp:contentType, dpp:url, dpp:language,
    dpp:DigitalProductPassport;
  dct:creator "European Commission";
  dct:license <https://interoperable-europe.ec.europa.eu/collection/eupl/eupl-text-eupl-12> .

dpp:digitalProductPassportId a skos:Concept;
  skos:notation "digitalProductPassportId";
  skos:prefLabel "digitalProductPassportId"@en;
  skos:inScheme <https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary>;
  dct:source <http://data.europa.eu/eli/reg_impl/2026/1778/oj>;
  dct:references
<https://standards.cencenelec.eu/ords/f%3Fp%3DCEN:110:::::FSP_PROJECT%2CFSP_ORG_ID:80714%2C3342699%
26cs%3D12990D18E0F87D7768D50B938A2F4A4D5>;
  euvoc:status concept-status:CURRENT;
  dct:created "2026-10-30"^^xsd:date;
  skos:topConceptOf <https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary>;
  skos:definition "Identifier of an instance of the digital product passport."@en ..
```

3.3 Product Group vocabulary

The **Product Group Vocabulary** (e.g. Battery vocabulary, Textile vocabulary, etc) is the product-specific taxonomy in the inventory, published as its own concept scheme. This vocabulary defines the data points specific to the product - extending the DPP vocabulary the way the UML diagram showed earlier.

Its scheme's source points to the corresponding Regulation (e.g. Regulation (EU) 2023/1542 – the EU Battery Regulation -).

Structurally, this vocabulary reuses the same modelling pattern described earlier – one *skos:ConceptScheme*, many *skos:Concept* entries, each with a notation, label, definition, scheme membership, source, and status – but it also has internal hierarchy. Rather than treating every data point as an independent top-level concept, related fields are grouped under a broader parent concept using

skos:broader. The first such grouping in the file covers manufacturer-related data.

```
@prefix bat: <https://repository.product-passport.ec.europa.eu/product/batteries/vocabs/battery-
vocabulary#> .

<https://repository.product-passport.ec.europa.eu/product/batteries/vocabs/battery-vocabulary>
  a owl:Ontology, skos:ConceptScheme;
  owl:versionIRI <https://repository.product-
passport.ec.europa.eu/product/batteries/vocabs/battery-vocabulary/1.0.0>;
  skos:notation "battery-vocabulary";
  skos:prefLabel "Battery Digital Product Passport Vocabulary"@en;
  dct:title "Battery Digital Product Passport Vocabulary"@en;
  skos:definition "Vocabulary of concepts used to support the battery digital product passport data
model. Regulation (EU) 2023/1542."@en;
  dct:source <http://data.europa.eu/eli/reg/2023/1542/2026-08-13>;
  dct:created "2026-10-30"^^xsd:date;
  owl:versionInfo "1.0.0";
  dct:license <https://interoperable-europe.ec.europa.eu/collection/eupl/eupl-text-eupl-12>;
  dct:creator "European Commission", "GROW-DIGITAL-PRODUCT-PASSPORT@ec.europa.eu";
skos:hasTopConcept bat:manufacturerIdentification.

bat:registrantIdentity a skos:Concept;
  skos:notation "registrantIdentity";
  skos:prefLabel "Identity of who is registering and/or is responsible for the battery
passport"@en;
  skos:inScheme <https://repository.product-passport.ec.europa.eu/product/batteries/vocabs/battery-
vocabulary>;
  dct:source <http://data.europa.eu/eli/reg/2023/1542/2026-08-13>;
  euvoc:status concept-status:CURRENT;
  dct:created "2026-10-30"^^xsd:date;
  skos:topConceptOf <https://repository.product-
passport.ec.europa.eu/product/batteries/vocabs/battery-vocabulary>;
  skos:scopeNote "Regulation (EU) 2023/1542 - Article 77 (3) - as defined in Article 77 (4).
Public. EV: Mandatory, LMT: Mandatory, Industrial: Mandatory."@en;
  skos:definition "Identity of who is registering and/or is responsible for the battery passport
[name]"@en;
  dct:accessRights dpptype:publicType .

bat:manufacturerIdentification a skos:Concept;
  skos:notation "manufacturerIdentification";
  skos:prefLabel "Manufacturer identification"@en;
  skos:inScheme <https://repository.product-passport.ec.europa.eu/product/batteries/vocabs/battery-
vocabulary>;
  dct:source <http://data.europa.eu/eli/reg/2023/1542/2026-08-13>;
  euvoc:status concept-status:CURRENT;
  dct:created "2026-10-30"^^xsd:date;
  skos:topConceptOf <https://repository.product-
passport.ec.europa.eu/product/batteries/vocabs/battery-vocabulary>;
  skos:scopeNote "Regulation (EU) 2023/1542 - Annex VI Part A 1. Public. EV: Mandatory, LMT:
Mandatory, Industrial: Mandatory."@en;
  skos:definition "Manufacturer name, registered trade name or registered trademark [at least one
of]"@en;
  dct:accessRights dpptype:publicType .
```

In the previous example, there is exactly one *skos:ConceptScheme* (*battery-vocabulary*) and every concept in the excerpt points back to it via *skos:inScheme*, following the same one-scheme-many-concepts pattern as the DPP vocabulary.

What's new is the relationship between the concepts themselves. *manufacturerInformation* is marked *skos:topConceptOf* - it sits at the top of the hierarchy, representing the general notion of "who the manufacturer is". *manufacturerName* is not top concepts; instead, each carries *skos:broader manufacturerInformation*, marking them as **narrower** concepts nested under that parent. The full grouping (not shown in full here) also includes other manufacturer data information.

This hierarchy matters for two reasons. Semantically, it captures that these fields aren't independent facts but different facets of a single real-world entity - the manufacturer - worth stating explicitly rather than leaving implicit in a shared naming prefix.

Chapter 4. SHACL Shapes

Just as every SKOS taxonomy, every SHACL shape file is built from the same small set of recurring elements. Understanding this shared pattern once makes every shape file in the inventory readable, regardless of which part of the data model it validates.

Element	Description
Node Shape	Each shape file defines one or more sh:NodeShape – the top-level unit of validation. A node shape describes the rules that apply to a particular “kind” of resource in the data (e.g., a DPP instance, a battery passport, an EV-specific battery record).
Target	A sh:targetClass (or sh:targetNode/sh:targetSubjectsOf) tells the shape which data it applies to – typically the RDF class corresponding to the vocabulary concept being validated. Without a target, a shape has rules but nothing to apply them to.
Property Shape	Within a node shape, each individual field being validated is expressed as a sh:property block – a nested property shape.
Path	Every property shape names the exact property it constrains via sh:path – pointing to the corresponding vocabulary term (e.g., dpp:granularity, bat:manufacturerName). This is what ties the validation layer directly back to the SKOS taxonomies: a shape reuses the notations already defined in the vocabularies.
Cardinality	sh:minCount and sh:maxCount specify how many times a property may or must appear.
Datatype / node kind constraints	sh:datatype (e.g., xsd:dateTime, xsd:string) or sh:nodeKind constrains what kind of value is allowed – ensuring, for instance, that dpp:lastUpdated is a real datetime rather than free text.
Value constraints	sh:in, sh:pattern, sh:minLength/sh:maxLength, or sh:hasValue restrict the actual allowed values — most importantly, sh:in is how a shape enforces that a property like dpp:granularity or bat:batteryCategory must draw from the corresponding SKOS vocabulary’s concepts, rather than accepting arbitrary strings.
Messaging	sh:message define how a failed constraint is reported.
Version info	owl:versionInfo is a simple annotation property carrying a human-readable version label as a literal - typically the SemVer string itself (e.g. "2.1.0").
Version IRI	owl:versionIRI is a stronger, structural annotation that assigns a distinct, dereferenceable IRI to a specific version of an ontology or vocabulary. It the IRI described in the versioning section.

Additionally, all SHACL shape files declare the namespaces following the URI structure pattern:

- Core URI:

```
https://repository.product-passport.ec.europa.eu/core/shapes/core-shapes
```

- Product URI:

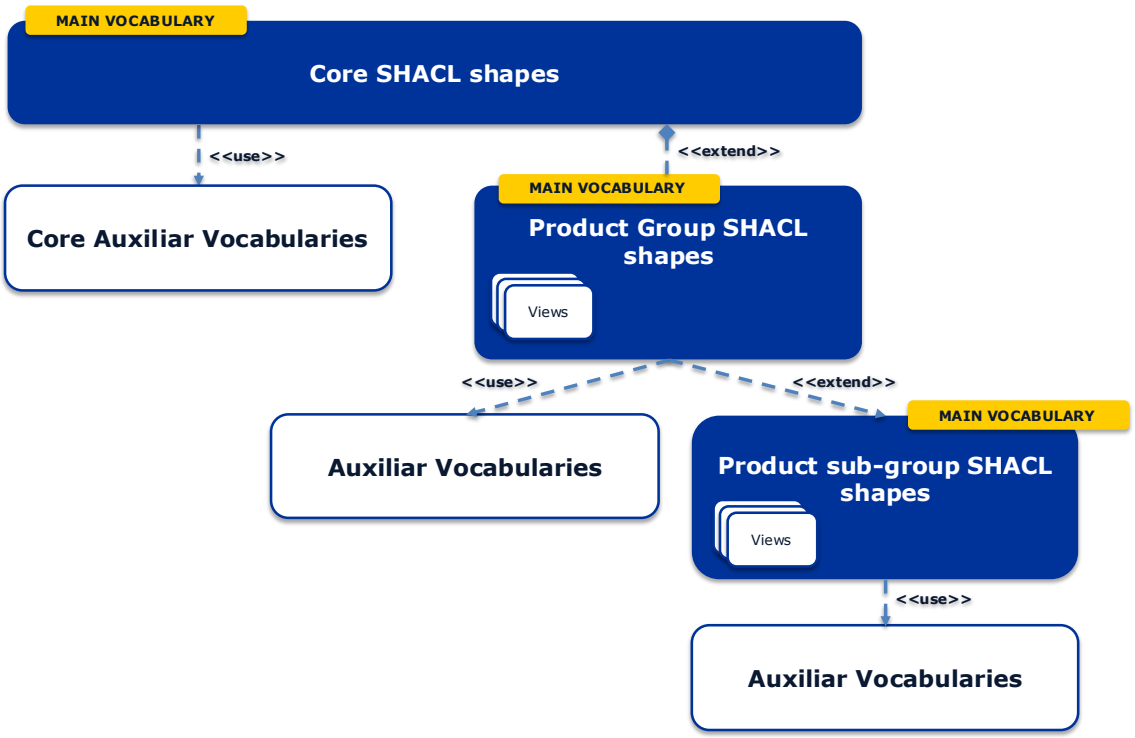
```
https://repository.product-passport.ec.europa.eu/product/{product-name}/shapes/{name}-shapes
```

- Product subgroup URI:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/subgroup/{subgroup-name}/shapes/{name}-shapes
```

4.1 SHACL shapes structure

This diagram illustrates the SHACL shape architecture of the DPP framework – showing how validation rules are organized into a layered hierarchy that mirrors the SKOS taxonomy structure described earlier.



At the top, **Core SHACL Shapes** is the core master SHACL shape, validating the baseline structure common to any DPP. It has a <<use>> dependency on the Auxiliar Vocabularies, meaning it constrains some properties to only accept values from these vocabularies.

Below it, **Product Group SHACL Shapes** is linked to the DPP shape via <<extend>>, meaning it inherits and builds on the baseline DPP validation rules while adding product-specific ones. It also has a <<use>> dependency on the auxiliary vocabularies. Last, **Product sub-group SHACL shapes**, which extends the Product Group validation rules. Additionally, each Product group SHACL shapes and Product sub-group SHACL shapes could be available per view.

4.2 Constraint types

Beyond the general shape of a SHACL what really drives the validation logic is the specific **constraint types** used inside each property shape. All shapes use a mix of them: **cardinality** (sh:minCount/sh:maxCount) to control how many times a field may appear, **datatype and node kind** constraints to control what kind of value is allowed, **SPARQL constraints** (sh:sparql) for rules a built-in keyword can't express – like checking a value against another vocabulary or a custom pattern – and **logical constraints** (sh:or) for rules requiring a choice between alternatives rather than one fixed requirement. Each is illustrated below with an example straight from the files.

Cardinality constraints – sh:minCount / sh:maxCount

These constrain **how many times** a property may appear. Used to distinguish mandatory single-value fields, optional fields, and repeatable fields. By default is set to optional and repeatable field (minCount is set to 0, and maxCount to many).

This example means the DPP identifier must appear **exactly once** - neither missing nor repeated.

```
sh:property [
    sh:path dpp:digitalProductPassportId ;
    sh:minCount 0 ;
    sh:maxCount 1 ;
    …
]
```

Here only sh:maxCount is set – the field is **optional but not repeatable**: it may be absent, but never appear more than once.

```
sh:property [
    sh:path dpp:uniqueProductIdentifier ;
    sh:maxCount 1 ;
    …
]
```

Datatype constraints – sh:datatype

These constrain **what kind of literal value** is allowed, tying a property directly to an XSD or RDF datatype.

```
sh:property [
    sh:path dpp:lastUpdate ;
    sh:datatype xsd:dateTime ;
    …
]
```

Ensures the value is a real, parseable datetime — not a free-text date string.

Node kind constraints - sh:nodeKind

Rather than constraining a literal's datatype, this constrains **what type of RDF term** a value must be – a literal, an IRI, or a blank node.

```
sh:property [
    sh:path dpp:granularity ;
    sh:nodeKind sh:IRI ;
    …
]
```

This requires dpp:granularity's value to be an IRI than a plain string literal – enforcing that granularity must point to a real vocabulary term, not just contain the word "item" as text.

SPARQL-based constraints - sh:sparql

Used when the built-in SHACL constraint types aren't expressive enough – typically to check a value against another vocabulary or a custom pattern.

This example enforces a format pattern on a language tag rather:

```
sh:property [
        sh:path bat: extinguishingAgent ;
        sh:message "SEM-COM-MSG-012";

        sh:sparql [
            a sh:SPARQLConstraint ;
            sh:select """
                PREFIX bat: <https://repository.product-
passport.ec.europa.eu//product/batteries/vocabs/battery-vocabulary#>
                PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

                SELECT $this (?value AS ?path)
                WHERE {
                    $this bat:usableExtinguishingAgent ?value .
                    BIND(lang(?value) AS ?langTag)
                    FILTER(!regex(?langTag, "^[A-Za-z]{2}-[A-Za-z]{2}$"))
                }
            """ ;
        ] ;
] .
```

This checks that the language tag on e*xtinguishingAgent* follows the xx-YY pattern (e.g. en-GB) required by

ISO 639/ISO 3166 - a constraint no built-in SHACL keyword can express directly, so SPARQL is used instead.

Logical constraints - sh:or

Used to express a choice between alternatives – at least one of several properties must be satisfied, rather than requiring all of them.

```
sh:message "SEM-BAT-MSG-001" ;

sh:or (
    [
        sh:property [
            sh:path bat:manufacturerName ;
            sh:minCount 1;
        ]
    ]
    [
        sh:property [
            sh:path bat:manufacturerTradeName ;
            sh:minCount 1
        ]
    ]
    [
        sh:property [
            sh:path bat:manufacturerTrademark ;
            sh:minCount 1
        ]
    ]
)
```

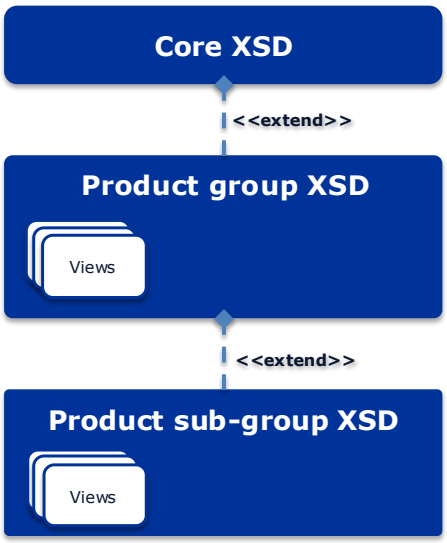
Here, neither *manufacturerName*, *manufacturerTradeName* nor *manufacturerTrademark* are individually mandatory – but this sh:or block enforces that at least one of the three must be present.

Chapter 5. Data Models

The final group of artifacts in the inventory governs how DPP data is structured and exchanged in concrete formats.

5.1 XSD

This diagram illustrates the XSD architecture of the DPP - showing the layered hierarchy that mirrors the structure described for SKOS taxonomies and SHACL shapes.



Each <<extend>> relationship means the lower schema inherits everything defined above it and layers its own.

The XSD files declare the namespaces following the URI structure pattern:

- Core URI:

```
https://repository.product-passport.ec.europa.eu/core/schemas/core.xsd
```

- Product URI:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/schemas/{name}.xsd
```

- Product subgroup URI:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/subgroup/{subgroup-name}/schemas/{name}.xsd
```

In XSD, this extension is implemented using *xs:import* and *xs:extension*, pointing at the schema one level up in the hierarchy:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:dpp="https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary#"
           targetNamespace="https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary#"
           elementFormDefault="qualified">
```

```
  <xs:import namespace="https://repository.product-passport.ec.europa.eu/core/vocabs/core-
vocabulary#" schemaLocation="https://repository.product-
passport.ec.europa.eu/core/schemas/core.xsd"/>
  <xs:element name="DigitalBatteryPassport" type="dpp:CoreXsdSchema"/>

    <xs:complexType name="CoreXsdSchema">
      <xs:complexContent>
        <xs:extension base="dpp:CoreXsdSchema">
          <xs:sequence>
            //New elements
            …
          </xs:sequence>
```

xs:import imports every type, element, and group declared in the referenced schema and allows to redefine selected components of it. Within the *xs:extension* block, a type can be restated in terms of itself: a product group schema can take a core type and extend it with additional elements, or restrict it to a narrower set of permitted values, while keeping the same type name.

This is an XML instance document for a Digital Battery Passport of product group Batteries and product sub-group EVB (Electric Vehicle Battery). Its job is to demonstrate the three-tier XSD hierarchy from the diagram in practice: even though every element here is written under a single *core:* namespace, each element actually "belongs" to a different layer of the schema hierarchy, and the document is validated against all three at once.

```
<dpp:DigitalBatteryPassport
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:dpp="https://repository.product-passport.ec.europa.eu/core/vocabs/core-
       vocabulary#"
       xsi:schemaLocation="https://repository.product-
       passport.ec.europa.eu/core/schemas/core.xsd/">

       //Core Elements
       <dpp:contentSpecificationIds>https://repository.product-
       passport.ec.europa.eu/core/1.0.0</dpp-contentSpecificationIds>

       //Product group elements
       <dpp:manufacturerName>ExampleCorp</dpp:manufacturerName>
       <dpp:batteryCategory>EVB</dpp:batteryCategory>

       // Product subgroup elements
       <dpp:stateCertifiedEnergy>90</dpp:stateCertifiedEnergy>
</dpp-scheme:DigitalBatteryPassport>
```

xmlns:dpp prefix used throughout the document to the core namespace, *https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary#* .

xsi:schemaLocation is a namespace location pair.

This is the key point: the instance document doesn't point directly at the Core XSD - it points at the most specific schema, *battery.xsd* (the Batteries product schema). Because *batteries.xsd* uses *xs:import* to pull in and extend the Product group XSD, which itself redefines the Core XSD, validating against batteries.xsd transitively validates the document against all three layers at once - core, product group, and sub-group - through that chain of redefinitions. The instance document itself doesn't need to know or care that three separate files are involved; it only references the endpoint of the chain.

5.2 JSON schema

This diagram illustrates the layered hierarchy in the JSON Schema architecture.

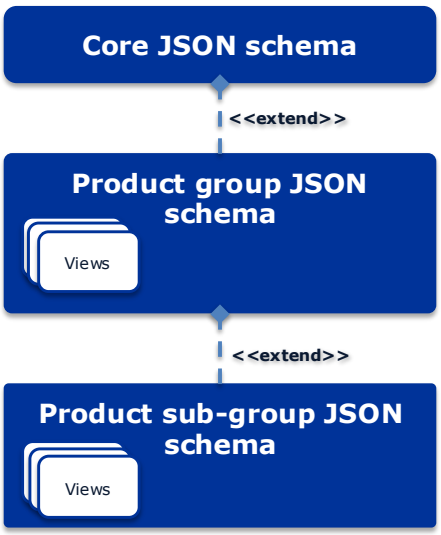


Figure 5.1 JSON Schema hierarchy

The JSON Schema files declare the namespaces following the URI structure pattern:

- Core URI:

```
https://repository.product-passport.ec.europa.eu/core/{SemVer}/schemas/core.schema.json
```

- Product URI:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/{SemVer}/schemas/{view}-{product-name}.schema.json
```

- Product subgroup URI:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/subgroup/{subgroup-name}/{SemVer}/schemas/{view}-{name}.schema.json
```

JSON Schema composes schemas from different layers using *$ref*, which requires a document to satisfy *every* schema listed, simultaneously:

```
{
  "$id": "https://repository.product-passport.ec.europa.eu/product/batteries/1.0.0/schemas/pub-batteries.schema.json",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Battery Product - Public view",
  "description": "JSON Schema for the public properties of the Battery Product representation.",
  "allOf": [
    { "$ref": "https://repository.product-passport.ec.europa.eu/core/1.0.0/schemas/core.schema.json" }
  ],
  "properties": {
    "registrantIdentity": {
      "type": "string",
      "minLength": 1,
      "message": {
        "type": "SEM-BAT-MSG-089"
      }
    },
….
```

Each *$ref* pulls in one layer of the hierarchy. A JSON instance needs to validate against *core.json* - this

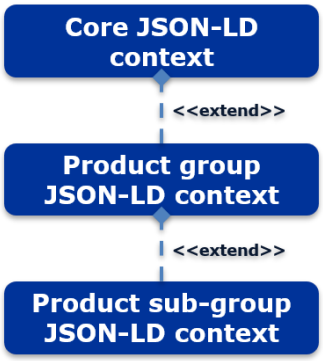
enforces all three layers at once.

```
{
  "digitalProductPassportId": "https://www.example.org/dpp/1234545",
  "manufacturerName": "ExampleCorp"}
```

There is one flat JSON object, no namespaces to manage, yet three distinct sets of validation rules - core, product group, and sub-group - applied together through schema composition. This keeps the JSON Schema layer structurally aligned with the XSD, SHACL, and SKOS layers already described, even though each format expresses "extension" through its own native mechanism.

5.3 JSON-LD context

Same as the previous data models, the diagram illustrates the layered hierarchy in the JSON-LD context architecture.



The JSON-LD context files declare the namespaces following the URI structure pattern:

- Core URI:

```
https://repository.product-passport.ec.europa.eu/core/contexts/core.jsonld
```

- Product URI:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/contexts/{name}.jsonld
```

- Product subgroup URI:

```
https://repository.product-passport.ec.europa.eu/product/{product-name}/subgroup/{subgroup-name}/contexts/{name}.jsonld
```

JSON-LD contexts follow the same three-tier pattern, composed here using @import, which pulls in the term mappings defined by a context one level up in the hierarchy:

```
{
  "@context": {
    "@import": "https://repository.product-passport.ec.europa.eu/core/contexts/core.jsonld",
    "@protected": true,
    "dpp": "https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary#",
    "bat": "https://repository.product-passport.ec.europa.eu/product/batteries/vocabs/battery-vocabulary#",
    "xsd": "http://www.w3.org/2001/XMLSchema#",
    "rdf": "http://www.w3.org/1999/02/22-rdf-syntax-ns#",
    "@base": "https://repository.product-passport.ec.europa.eu/core/vocabs/core-vocabulary#",
    "manufacturerName": {
```

```
      "@id": "bat:manufacturerName",
      "@type": "xsd:string"
    }
…
```

@import brings in every term mapping already defined in *core.jsonld*; it inherits them, and only declares the terms specific to this layer.

```
{
    "@context": "https://repository.product-passport.ec.europa.eu/core/contexts/core.jsonld",
    "@id": "https://www.example.org/dpp/1234545",
    "@type": "dpp:DigitalProductPassport",

    "digitalProductPassportId": "https://www.example.org/1234545",
    "manufacturerName": "ExampleCorp"}
```

Chapter 6. JTC 24 Alignment

The defined DPP data models are aligned with the CEN-CENELEC JTC 24 standard (EN 18223:2026).

One deviation exists, concerning how multilingual values are represented in XML. **Annex B** of EN 18223:2026 defines the normative pattern as a nested structure, with the value and its language declared as separate child elements:

```
<specA:productDescription>
    <specA:MultiLanguageValue>
        <specA:value>One Thing</specA:value>
        <specA:language>en-IE</specA:language>
    </specA:MultiLanguageValue>
</specA:productDescription>
```

In this inventory's implementation, the same information is instead expressed using the standard XML xml:lang attribute directly on the element:

```
<specA:productDescription xml:lang="en-IE">One thing</specA:productDescription>
```

Both forms carry the same information - a value and its language - but the inventory's approach uses XML's native language-tagging mechanism rather than introducing a dedicated nested type for it, keeping the representation simpler and more consistent with how language-tagged literals (rdf:langString) are already handled elsewhere in the SHACL and JSON-LD layers. This is flagged as the one known point of departure from the JTC 24 normative examples, rather than a silent inconsistency.

Chapter 7. Battery DPP Examples

The sections above describe how each artifact is structured and how they relate to one another. The examples are included in the ZIP file accompanying this document.

Each example is annotated to trace individual values back to the concept, constraint, or schema that defines them - showing, concretely, how the core, product, and sub-group layers combine in a real instance, how contentSpecificationIds declares the exact releases it conforms to, and how the same underlying data can be represented consistently as RDF, XML, or JSON.

All samples start with the core DPP fields - identifier, unique product identifier, granularity (Model), schema version, status, last-updated timestamp, and economic operator/facility IDs - followed by two contentSpecificationIds values, satisfying the "at least two" restriction described earlier (one presumably resolving to the core release, one to the battery product release).

The rest of the document adds battery product-group fields: manufacturer details (name, trade name, trademark, address, website, email), batteryCategory (EVB, marking this as an EV battery), and battery-specific attributes - model identifier, serial number, place/date of manufacture, mass, chemistry (LFP), extinguishing agent, and critical raw materials.

Last, it includes Electric Vehicle Battery sub-group-specific field (stateCertifiedEnergy).